GUIA VISUAL 020

PERMISSÕES DE ARQUIVO

Quem pode ler, escrever e executar — e por que `chmod 777` não resolve.

020
GUIAS LINUX

ANATOMIA DA SAÍDA DO `LS -L`

-	r w -	r - -	r - -
TIPO	DONO	GRUPO	OUTROS
`arquivo`	`6`	`4`	`4`

r = ler (4)   **w** = escrever (2)   **x** = executar (1)   → `-rw-r--r-- = 644`

A REGRA QUE QUASE NINGUÉM APRENDE

SÓ UMA CLASSE SE APLICA

O Linux checa na ordem: é o **dono**? senão, está no **grupo**? senão, **outros**. Para na primeira que casa.

EM ARQUIVO

r lê o conteúdo · **w** altera o conteúdo · **x** executa como programa.

EM DIRETÓRIO

r lista nomes · **w** cria/remove itens · **x** *atravessa* o diretório. Sem **x**, nada dentro é acessível.

AS DUAS NOTAÇÕES DO `CHMOD`

SIMBÓLICA

- ✔ Alvo: `u` dono · `g` grupo · `o` outros · `a` todos
- ✔ Operador: `+` adiciona · `-` remove · `=` define
- ✔ Ex.: `chmod u+x script.sh`

OCTAL

- ✔ Soma dos bits por classe: **r=4, w=2, x=1**
- ✔ Três dígitos: dono · grupo · outros
- ✔ Ex.: `chmod 755 script.sh`

NA PRÁTICA

OCTAL	SIMBÓLICO	USO TÍPICO
`644`	`-rw-r--r--`	Arquivo comum (todos leem, só o dono edita)
`600`	`-rw-------`	Arquivo privado — chave SSH, segredo
`755`	`-rwxr-xr-x`	Script/binário e diretórios navegáveis
`700`	`-rwx------`	Diretório pessoal, script só seu (`~/.ssh`)

chmod 777 não é solução de nada. Abre leitura, escrita e execução para *qualquer* usuário da máquina. Se o acesso falha, o culpado quase sempre é a falta de `x` em um diretório do caminho — não a permissão do arquivo.

⚡ RESUMO RÁPIDO

- ✔ Três classes (dono/grupo/outros), três bits (rwx —

🐉 NO KALI

O Kali roda muita coisa como **root**, que ignora os bits de